

Scripts Showcase

Operation fsoc: Case Study

Gurmann Basran

1 What the project ships

A small set of production monitoring and auto-healing scripts runs across the infrastructure, totalling a little over a thousand lines of Bash. Each one has a specific detection target, an alert path, and a recovery action. This document describes each at the goal level only. What I specifically do not include: the code itself, the layer each script watches, the tool it is integrating with, the thresholds it triggers at, the specific invariants it verifies, and the mechanism it uses to alert. All of those details would help someone trying to evade detection against my specific deployment, and I do not think the educational value of publishing them outweighs that cost. What is preserved is the reasoning, which is what transfers to another project anyway.

2 Defensive-chain invariant checker

Some tools can be “running” without actually providing the protection they were deployed for. Their process is alive, the basic healthcheck passes, and yet a specific invariant that makes the protection meaningful has silently regressed. This script runs on a tight interval and tests the invariant directly: not “is the service responding?” but “is the service responding *correctly*, in a way that depends on configuration I care about?” When the answer is no, the script both alerts and applies the fix. The design lesson is that liveness checks and correctness checks are different checks; naive healthchecks answer the liveness question and miss the correctness question entirely.

3 Out-of-band perimeter monitor

Any script running on the perimeter device cannot alert if the perimeter device itself is the thing that is down. A single-host monitor creates a “who watches the watcher” failure: the operator finds out by not getting the morning digest, which is hours too late. The monitor for this component runs on a different physical host with independent reachability. The design lesson is that every safety-critical check has to run from a host that does not depend on the thing being checked.

4 Remote-access liveness monitor

Remote-access protocols can be “up” in the sense that the daemon is listening while the actual connections from trusted devices have silently gone away. This script reads the connection-state telemetry through the component’s API and alerts if any expected peer has not completed activity within a defined freshness window. The design lesson is the same as the previous script: the correct question is whether the protocol is *functioning at the level I care about*, not whether the daemon is running.

5 Privileged-access real-time notifier

Every successful privileged login on every host pushes a real-time notification to my phone with the user, the source IP, and a geo-lookup. Alerts fire during login, not during log review. If I am starting a session from a coffee shop I expect the alert; if the alert arrives while I am asleep that is a page-one incident and I want to know about it immediately, not in the morning. The design lesson is that real-time notification on successful authentication beats daily log review for every metric that matters, because the point of the alert is to catch the event while it is still happening.

6 Daily state digest

Once a day, the infrastructure produces a summary of the previous twenty-four hours: authenticated sessions per host, detection-layer alert counts by severity, behavioral-defense actions, certificate expiry countdowns, disk utilisation per volume, backup status per machine, ingestion health across the logging spine, pending update notices. The content is not actionable in real time but it is the state snapshot I want in my inbox every morning. The design lesson is that a digest that *fails* to arrive is itself a signal; a separate “digest did not generate” watchdog catches that failure, because silent scripts are worse than noisy ones.

7 Image update watcher (manual-apply)

Unattended image updates are disabled. The watcher runs in monitor-only mode and generates an alert when new images are available; I apply updates manually after reading the release notes. The design rationale is that supply-chain attacks against public image registries have happened more than once in recent years, and unattended updates convert every image in the registry into a one-hop attack surface. The design lesson is that the default “auto-update everything” posture trades a small, bounded cost (manual review time) for an unbounded one (a catastrophic supply-chain event), and I choose the bounded cost.

8 Configuration export via least-privilege API

One of the infrastructure components holds the configuration that would be painful to lose. A daily automated job exports that configuration through the component’s API using a read-only, source-locked, scope-minimal credential. The export is checksummed and pushed to off-host encrypted storage. The design lesson is that backup paths should use the least privilege credential the job can possibly function with, because a backup credential that leaks should only ever enable reading what the backup already reads.

9 Remote workstation wake

From a trusted remote context, a script on an always-on inside host wakes a physical workstation that is otherwise asleep. Combined with the outbound reverse tunnel below, this is the “reach my

desktop while I am away” workflow. The design lesson is that the power-on signal should originate from a host already trusted inside the perimeter, with the remote operator reaching that trusted host through existing authorized access; no inbound port to the workstation is ever exposed.

10 Outbound-only reverse tunnel

A persistent outbound-initiated tunnel from a portable device to an always-on inside host, with automatic reconnection, lets the portable device be reached from the inside host on a chosen port. The perimeter never exposes an inbound port to that device. The design lesson, which is probably my favourite pattern in the whole project, is that every path that has to exist should be outbound from the more-trusted side of the boundary, never inbound from the less-trusted side. Mobile devices reach home; home does not expose itself to the internet to make that work. The same reasoning underpins modern zero-trust networking.

11 Design patterns common to every script

Some practices show up across every script in the set. None of them are innovations, but every one of them came from an incident where not doing it cost me something.

1. **Fail-loud execution.** Scripts abort on the first error, on unset variables, and on pipeline failures. No silent partial success.
2. **No inline credentials.** Every credential is pulled from an encrypted-at-rest file or a least-privilege API account; nothing is in the script text.
3. **Timeouts on every network call.** A script that hangs waiting for a dead endpoint is a script that will be noticed only after someone asks why the digest never arrived.
4. **Alert text is complete on its own.** A notification says what failed, what was expected, and what was observed. Future-me at 3 AM needs to understand what happened without opening the script.
5. **Idempotent auto-heal.** Fix actions are safe to run repeatedly against an already-healthy system.
6. **Least-privilege credentials per script.** Each script uses a different account, scoped to exactly what that script needs.
7. **Tested with deliberate breakage.** Every alert path was validated by manually triggering its failure condition before the script went into production. An untested alert path is an alert path that is not there.